wsdm-cup-multilingual-chatbot-arena

3rd place solution

Rank	3
Team	PLaMo 1000000B
Author	RihanPiggy
Topic ID	567584
Version	Original

Source: https://www.kaggle.com/competitions/wsdm-cup-multilingual-chatbot-arena/discussion/567584
Retrieved with Kaggle CLI on 2026-10-21.

Congratulations to all the winners! Thanks to the organizers for hosting such an interesting competition. LMSYS was my first LLM competition with a sliver medal. And I am very happy to be able to win a gold medal this time at WSDM Cup. Here Let me share my solution.

Training code: https://github.com/LIHANG-HONG/WSDM-Cup-3rd-place-solution
Inference code: https://www.kaggle.com/code/honglihang/wsdm-submission-nocot-public?scriptVersionId=228251584

TL;DR

- Reimplement the training pipeline of Eedi Rerank Model. Use AutoModelForCasualLM rather than AutoModelForSequenceClassification for faster inference with vllm.
- Qwen/Qwen2.5-14B-Instruct(with post-pretrain) + Phi4(without post-pretrain). Merge model trained with different seed.
- distillation with 72B Model as 1st place of LMSYS. But I found that the 72B model is not necessary. Using the logit of 14B model and perform "self-distillation" can reach the same CV score. Maybe it is the the effect of label cleaning brought by soft logit that really matters.
- Quantize with auto-round.
- vllm inference. For ensemble, first sort by token length, and then perform 25% TTA with Qwen2.5-14B. The rest of the inference time is used by Phi4.

Preprocess

Use fasttext to infer the language of the prompt.

Train part

I override the compute_loss function of SFTTrainer to calculate the loss only for Token "A" and "B".

```
 ():
    ():
       labels = inputs[]
       labels[labels > ] = -
       inputs[] = labels
       _, outputs = ().compute_loss(model, inputs, return_outputs=,
num_items_in_batch=num_items_in_batch)
       logits = outputs.logits
       shift_logits = logits[..., :-, :].contiguous()
       shift_labels = labels[..., :].contiguous()

       logits_target = []
       labels_target = []
        i  ((shift_labels)):
           lbl = shift_labels[i].cpu().numpy()
           target_idx = np.where(lbl != -)[][]
```

```
            logits_target.append(shift_logits[i][target_idx][:])
            labels_target.append(shift_labels[i][target_idx]-)


        logits_target = torch.stack(logits_target, dim=)

        labels_target = torch.tensor(labels_target).to(outputs.logits.device)
        loss = F.cross_entropy(logits_target, labels_target)
         return_choice_logit:
             (loss, outputs, logits_target)  return_outputs  (loss, logits_target)
        :
             (loss, outputs)  return_outputs  loss
```

The max length of prompt, response a and response b is limited to 2048. So the max length of the input is about 6144.

```
():
   preference_prompt_predict_template =

   preference_prompt_train_template = preference_prompt_predict_template +

   response_template =

   sep =
```

Post-Pretrain

I used the following data for post-pretrain. The post-pretrain share the same training pipeline with the training.

- ultrafeedback
- dataset collected by 2nd place of LMSYS

Training

I used the following data for training.

- LMSYS data
- LMSYS 33k
- WSDM Cup data

For teacher models(Qwen2.5-72B and Athene-V2-Chat), I used deepspeed to train on multiple GPUs during the New Year holiday. To avoid adjusting the learning rate, I simply set gradient_accumulation_steps to 1 to keep the batch size same as single GPU training.

CV(without distillation)

Basically, learning rate 10e-5 is good for 14B model. Loss spike starts to occur around 14e-5. For 72B model, a smaller learning rate is better.

No	Experiment	Fold	lr	Accuracy
1	Qwen2.5 14B(post-pretrain, only WSDM data)	0	7e-5	0.7055360462714315
2	-	0	8e-5	0.7007849617847552
3	-	0	9e-5	0.699958686221855
4	-	0	**10e-5**	**0.7051229084899814**
5	-	0	12e-5	0.7059491840528817
6	-	0	14e-5	0.5084693245197274
7	Phi4(only WSDM data)	0	7e-5	0.6983061350960545
8	-	0	8e-5	0.6965502995248916
9	-	0	9e-5	0.6995455484404048
10	-	0	10e-5	0.6966535839702541
11	-	0	12e-5	0.6973765750877918
12	-	0	14e-5	0.7016112373476554

CV(with distillation)

The best Temperature is 5.0(experiment 2~5) for me and the best soft loss weight is 0.9(experiment 2~7). Following the 1st place of LMSYS, I tried averaging KLDivLoss and CosineEmbeddingLoss, but the effect is not obivious. KLDivLoss is enough.

No	Experiment	Fold	Accuracy	T	distil_weight	lr
1	Qwen2.5 14B(baseline without distillation, only WSDM data)	0	0.7051229084899814	-	-	10e-5
2	Qwen2.5 14B(distillation experiments, only WSDM data)	0	0.7061557529436067	1	0.5	10e-5
3	-	0	0.7082214418508572	10	0.5	10e-5
4	-	0	0.7076017351786821	3	0.5	10e-5
5	-	0	0.7102871307581078	5	0.5	10e-5
6	-	0	0.7121462507746333	5	0.7	10e-5
7	-	0	**0.7134889485643462**	**5**	**0.9**	10e-5

No	Experiment	Fold	Accuracy	T	distil_weight	lr
8	Qwen2.5 14B(distillation with more teacher model, only WSDM data) - weight param set1	0	0.7168973352613096	5	0.9	10e-5
9	Qwen2.5 14B(distillation with more teacher model, only WSDM data) - weight param set2	0	0.7164841974798596	5	0.9	10e-5

Quantize

I used auto-round for quantization. Using a heavy quantize setting can surpress the accuracy drop. The quantize experiment is based on Qwen2.5 3B model.
My final quantize setting is seqlen=6144, nsamples=512, iters=3000.

No	Experiment	seqlen	nsamples	iters	Fold	Accuracy
1	baseline - no quantiz				0	0.681057
2	1 + lite setting	2048	256	500	0	0.674653
3	1 + official best setting	2048	512	1000	0	0.677752
4	3 + increase iter	2048	512	2000	0	0.678062
5	3 + increase iter	2048	512	3000	0	0.677649
6	4 + increase iter	2048	512	4000	0	0.673311 / 0.678785
7	3 + increase seqlen	4096	512	1000	0	0.676822
8	7 + increase iter	4096	512	2000	0	0.678888
9	8 + increase iter	4096	512	3000	0	0.673001
10	9 + increase iter	4096	512	4000	0	0.678888
11	10 + increase iter	4096	512	5000	0	0.680024
12	11 + increase seqlen	4096	512	6000	0	0.673104
13	7 + increase seqlen	6144	512	2000	0	0.678578
14	**13 + increase iter**	**6144**	**512**	**3000**	0	**0.680851 / 0.676513 / 0.681160 / 0.678578**
15	14 + increase iter	6144	512	4000	0	0.678682 / 0.674757

Inference part

I used vllm for inference. The inference part is nothing special, it is all about engineering. I used the following tricks.

- Dynamically change the inference time limit according to the test data size. For train phase, the inference time limit is set to 265min. For forecast phase, the inference time limit is set to 690min.
- Using T4x2 Environment to test the throughput of the model. The average throughput of the model is about 70000 token per minute. For TTA, thanks to the prefix cache of vllm, the throughput can reach about 74000 token per minute.
- Sort the test data by token length and perform TTA with Qwen2.5 14B model for the first 25% of the data.
- Monitor the inference time left and figure out the number of tokens that can be processed by Phi4 according to the throughput. About 60% of the test data can be processed in train phase.

Submission strategy

Basically, I choose the submission with the highest leaderboard score. Since the leaderboard score is not stable, I trained the same model with different seeds and linearly merged the weight of the model.
I find that phi4 performs well for short prompt but bad for long prompt in LB. So I changed the ensemble weight of Phi4 according to the token length.

1. Calculate the quantile of the prompt length of the test data. q1=0.25, q2=0.5, q3=0.75, q4=1.0
2. For token length <= q2, phi4 weight is set to 0.5.
3. For token length > q2 and <= q3, phi4 weight is set to 0.15.
4. For token length > q3, phi4 weight is set to 0.1.(Not used due to the inference time limit. Phi4 can only infer about 60% of the test data in train phase.)

What I tried but gave up

- Inspired by 1st place of Eedi, I distiled Qwen2.5-3b to generate CoT about how to solve the prompt(=request from user). The CV and LB get better by 0.001~0.002, but the inference time is too long and I frequently encountered timeout error.
- Tried Qwen2.5-14b distilled by deepseek and Qwen2.5-14b-1m. The LB is not good.

```
system_prompt_template =

user_prompt_template =
```

Supplemental Q&A

> **cloudwantyou@gmail.com · 2025-03-11T02:04:07.777000**
>
> Thanks for your explanation. We tried CoT too, but even after spending many resources on GPT-4o, the improvement was quite limited. I think we might need to adopt a reinforcement learning approach similar to DeepSeek-R1 for CoT data. anyway, congratulations!

> **RihanPiggy · 2025-03-11T02:12:14.377000**
>
> Thanks! And congratulations to you too!
> Exactly, distilation with SFT is not good enough. I used llama3-405B to generate the teacher CoT and trained Qwen2.5-3B. Besides little improvement, another issue for me is that after the SFT, the generation length of Qwen2.5-3B exploded compared to the one before distillation, leading to a significant inference time increase…

> **Miles · 2025-03-16T19:06:24.793000**
>
> Congrats! Could you give more details on how the TTA was performed? There are different ways to do it.

> **RihanPiggy · 2025-03-18T10:15:06.077000**
>
> Thank you. I simply swapped the response A and B. I sorted the dataframe according to the token length of the whole prompt and performed TTA to the first 25% of the data.
> I have made my inference code public. You can take a look at run_inference.py